

《相机标定学习手册》

该内容是公众号 计算机视觉life 旗下课程 《单目/鱼眼/双目/全景/阵列 相机标定：原理和实战》 的课件编纂成的课程手册。

说明：本文会随课程持续更新，由于笔者水平有限，编写过程难免存在错误或疏漏，读者如发现有错误，欢迎提出。

扫描一下，系统学习视觉/激光/多传感器融合SLAM、三维重建等精品课程：



扫描学习SLAM、三维重建

第六章 双目标定及立体视觉

1. 立体视觉

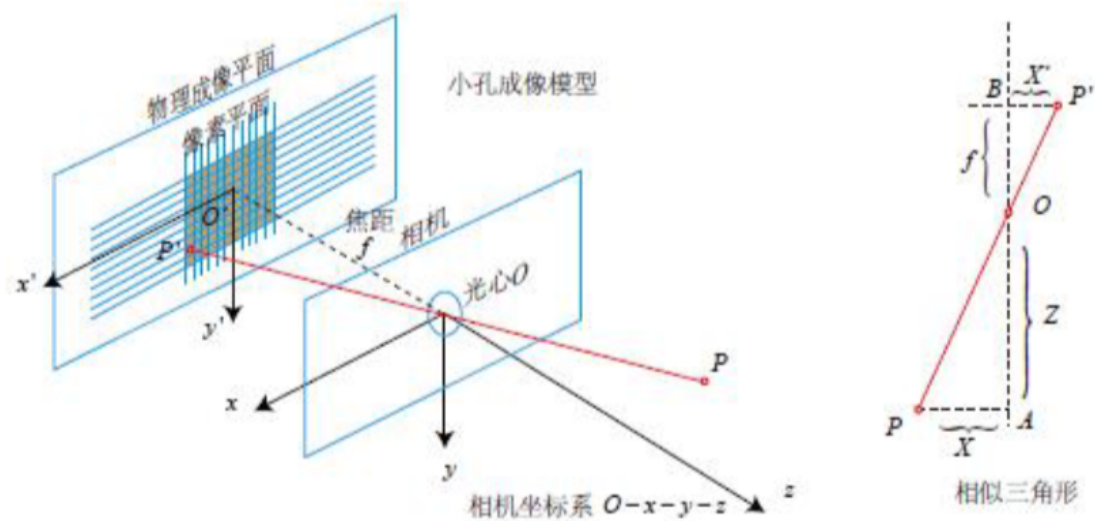
- 1.1 视差及深度恢复
 - 1.2 对极几何约束
 - 1.3 本征矩阵(Essential Matrix)
 - 1.4 基础矩阵(Fundamental Matrix)
 - 1.5 单应矩阵和基础矩阵差异和联系
 - 1.6 双目/立体校正(Stereo Rectify)(Bouguet 校正)
 - 1.6.1 再谈旋转矩阵
 - 1.6.2 构建校正矩阵
- ### 2. 双目标定代码讲解
- 2.1 部分知识
 - 2.1.1 相对旋转计算

第六章 双目标定及立体视觉

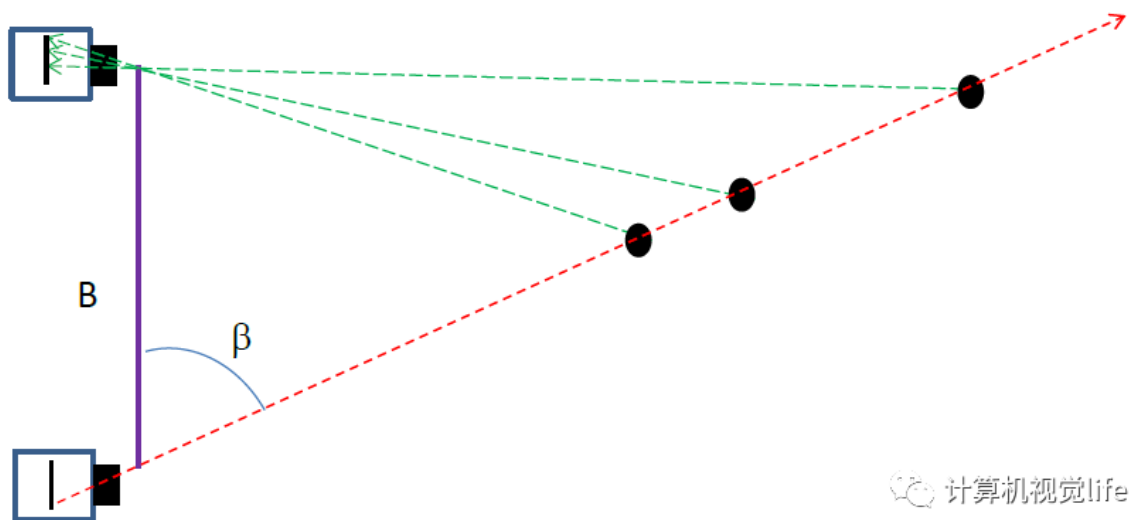
1. 立体视觉

1.1 视差及深度恢复

我们知道，单相机对于同一个角度入射光线上的任何物体，其成像位置都是一样的，也就是说投影本身会损失掉拍摄物体的 3D 信息，如果我们想恢复物体的深度信息呢？

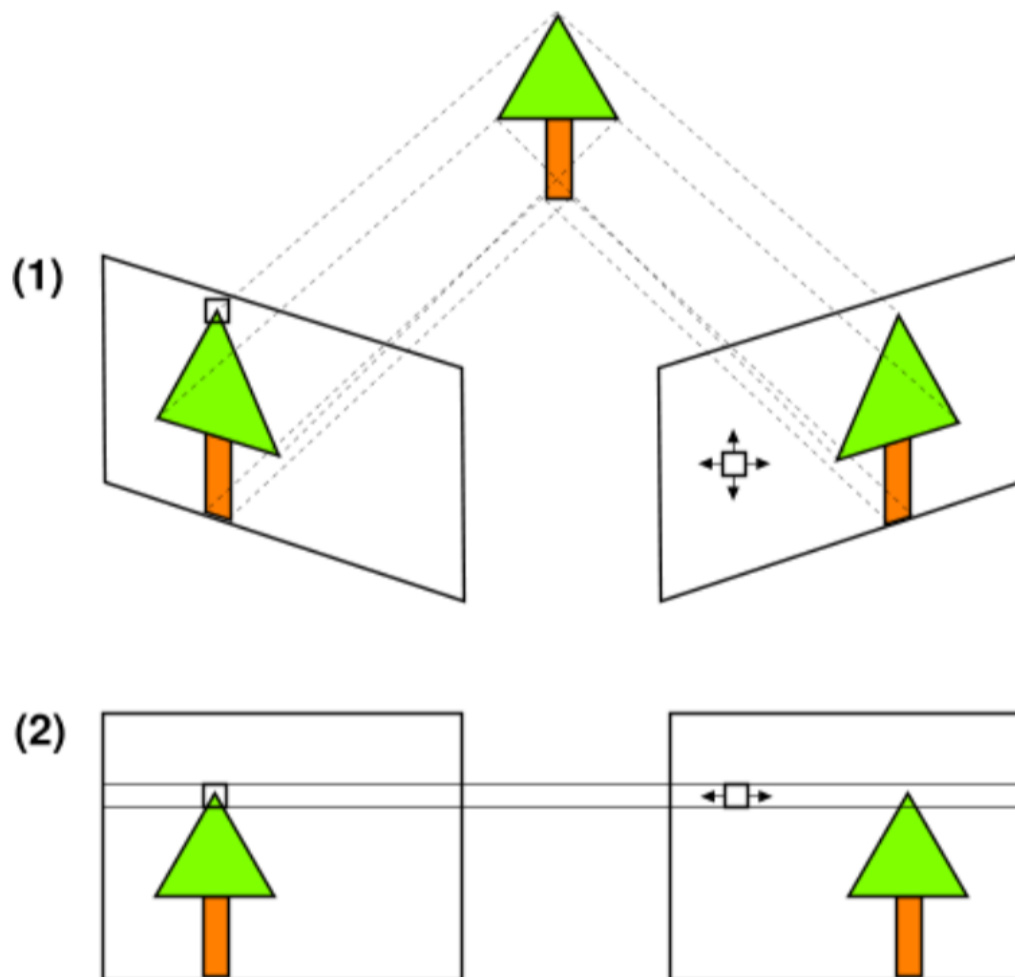


很简单，我们增加一个观察的维度，使用另一个相机观察同一个物体，通过两根入射光线，我们就可以确定具体物体的具体位置，如下图。

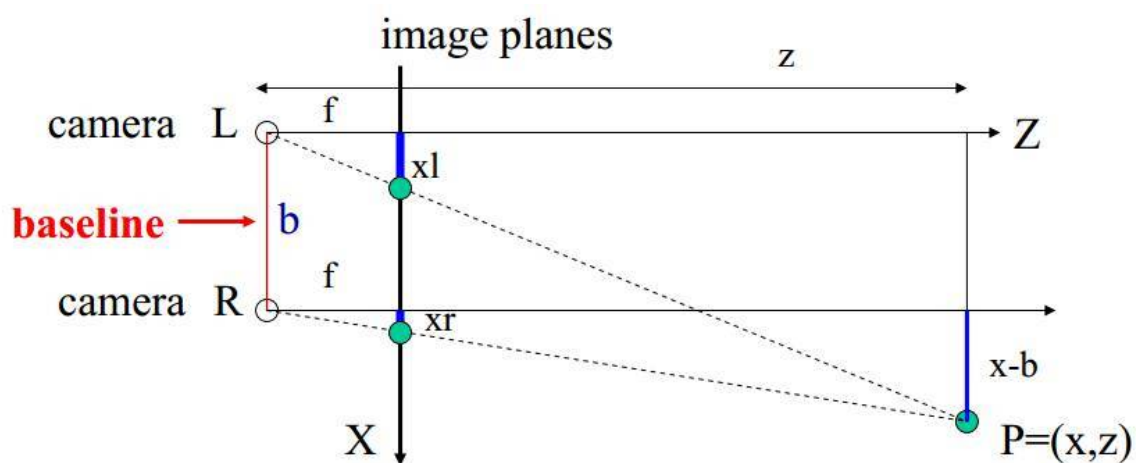


由于两个相机和观测点构成了一个三角形，所以我们也称这种为三角测距。当然，实际上三角测距并不止双目的情况，还有其他很多。

现在有了两个相机，我们来假定一种最理想的情况，两个相机焦距相等，的成像平面和光轴是完美平行的，并且其 x 轴方向也完美对齐，也就是两个相机不存在 y 方向的偏移，那么其会如下图



这种情况我们如何来恢复深度呢？



$$\frac{z}{f} = \frac{x}{x_l}$$

$$\frac{z}{f} = \frac{x-b}{x_r}$$

$$\frac{z}{f} = \frac{y}{y_l} = \frac{y}{y_r}$$

$$\text{Depth } z = f \cdot b / (x_l - x_r) = f \cdot b / d$$

$$x = x_l \cdot z / f \quad \text{or} \quad b + x_r \cdot z / f$$

$$y = y_l \cdot z / f \quad \text{or} \quad y_r \cdot z / f$$

Y-axis is perpendicular to the page.

我们把上式中的 b 称为基线， d 称为视差，其代表两个视角观测同一个点的差异。并且由上式也可以简单看出，物体越近，视差越大，反之亦然

直观感受视差，可以在眼前竖起一根手指，然后分别只用左右眼来观察它，可以看到手指在左右眼中的位置是不同的。

这里我们就有了一个恢复深度的简单思路，

1. 计算相机焦距和基线距离
2. 通过某种方法匹配左右相机匹配点，计算视差
3. 恢复深度

1.2 对极几何约束

但是上面只是一种理想的情况，实际上，我们的相机并不可能是完美平行的，更多可能是下面的状态

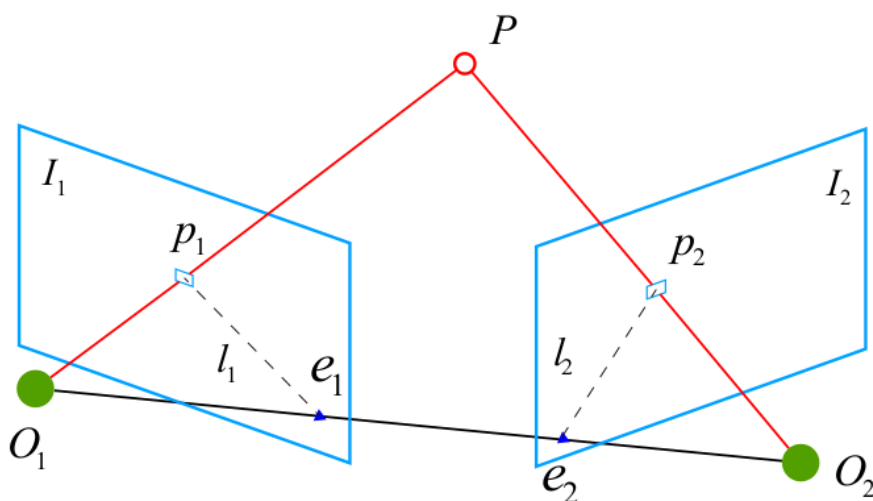


图 7-7 对极几何约束。

可以直觉的观察到，对于左相机成像平面上的任意一点 p_1 ，我们都可以通过沿着 O_1p_1 和 O_1O_2 构建的平面和右相机的交线方向来搜索其匹配点，从而来得到具体 3D 点 P 的位置，实际上，我们也把这个约束叫做 **对极几何约束**

为了更好的描述上述现象，我们引入几个概念：

1. 两相机中心的连线 O_1O_2 称为**基线(baseline)**
2. 观测物体平面 O_1PO_2 称为**对极平面(Epipolar Plane)**，任何包含**基线**的平面都是对极平面
3. 平面 O_1PO_2 与左右相机成像平面 $I_1 I_2$ 的交点，或者说两个相机光心与左右成像平面的交点 $e_1 e_2$ 称为**极点(Epipoles)**
4. O_1PO_2 与成像平面的交线 $l_1 l_2$ 称为**极线(Epipolar Lines)**

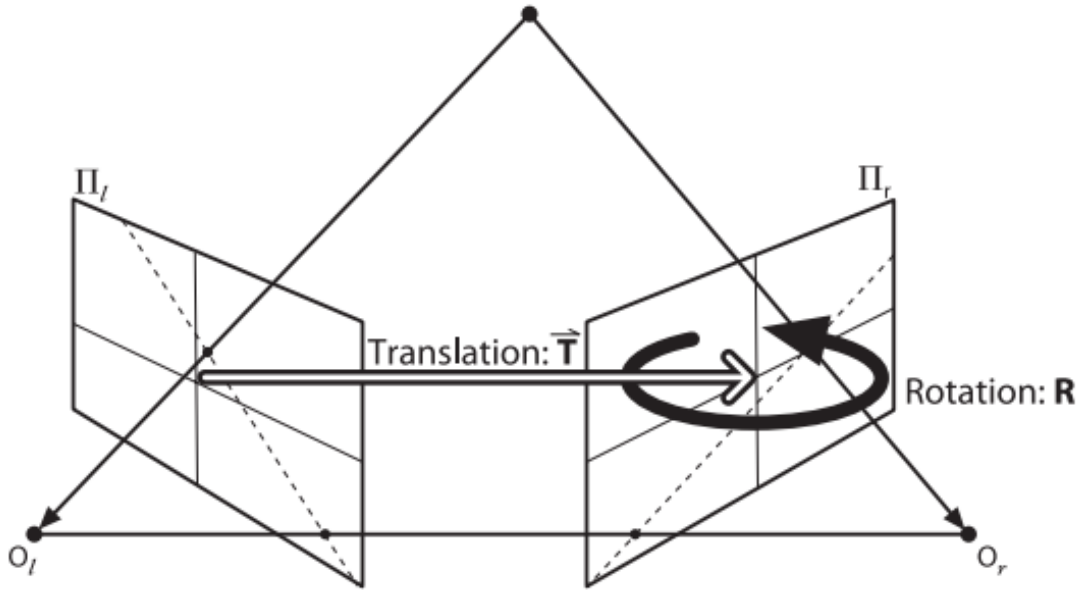
随着观测点 P 的上下移动，对极平面会绕着基线转动，但是极点的位置是固定不变的，或者说画面中任意一条极线都相交于极点位置。

极点位置并不一定都在画面中，比如上面提到的完美双目的极点位置实际上在无穷远处



1.3 本征矩阵(Essential Matrix)

上面的图让我们对对极约束有了直观的概念，我们来看看数学上如何来描述



假设我们观测点为 P ，在两个相机坐标系观测的点坐标分别为 P_l, P_r ，左相机到右相机的平移矩阵为 T ，相对旋转为 R ，那么有

$$P_r = RP_l + T \quad (175)$$

我们对上式的两侧分别叉乘一个 T

$$T \times P_r = T \times RP_l + T \times T = T \times RP_l \quad (176)$$

可以看出，其实上式的左边项为对极平面的**法线方向**，如果我们在上式左乘一个向量 P_r ，则有

$$P_r^T (T \times P_r) = P_r^T (T \times RP_l) \quad (177)$$

由于 P_r 一定和 $T \times P_r$ 垂直，则有

$$P_r^T (T \times RP_l) = 0 \quad (178)$$

并且我们之前讲解罗德里格斯旋转矩阵时也指出，叉乘等价于一个向量的反对称矩阵和另一向量进行矩阵乘法，因此有

$$P_r^T (T \times RP_l) = P_r^T [T_{\times}] RP_l = 0 \quad (179)$$

其中 $[T_{\times}]$ 为反对称矩阵，形式为

$$\mathbf{T}_{\times} = \begin{bmatrix} 0 & -T_z & T_y \\ T_z & 0 & -T_x \\ -T_y & T_x & 0 \end{bmatrix} \quad (180)$$

我们定义 $E = [T_{\times}]R$, 那么有

$$P_r^T E P_l = 0 \quad (181)$$

那么这个 E 我们就称为本征矩阵。我们计算的是 3D 点的相关关系，但是可以将点 P_r 和 P_l 归一化到 $Z = 1$ 的平面上, 分别为 $x_l x_r$, 这样也并不会破坏上述约束。

$$x_r^T E x_l = 0 \quad (182)$$

实际上, $l = E x_l$ 为右侧相机的极线, 同样, 也可以得到对应的左相机极线。

关于本征矩阵的一些讨论:

- 本征矩阵的秩为2, 因为反对称矩阵的秩为 2 , 旋转矩阵的秩为 3。其实也可以理解为, 已知点 x_l 和 E , 我们无法求解准确的 x_r , E 为奇异矩阵。
- 本征矩阵 SVD 分解中 Σ 的形式是 $diag\{a, a, 0\}$ 且 $a > 0$, 即其两个特征值相等。这个性质在 SLAM 中会被经常用到, 包括使用 E 进行分解求解 RT。
- 本征矩阵的**自由度是5**, 虽然其由具有 3 个自由度的平移和 3 个自由度的旋转组成, 但是由于我们上述的约束中乘以任何一个常量都成立, 因此其自由度实际上是 5。
- 本征矩阵求解最直接的是类似之前单应矩阵求解, 将上述约束展开, 使用最少八个点构建线性方程组从而求解。

1.4 基础矩阵(Fundamental Matrix)

实际上, 更多时候我们针对像素坐标系进行约束的, 需要将极线约束变换到像素平面上。假设左右相机的内参分别为 $K_l K_r$, 且我们认为画面无畸变, 则有

$$p_r^T K_r^{-T} E K_l^{-1} p_l = 0 \quad (183)$$

如果我们令 $F = K_r^{-T} E K_l^{-1}$, 就得到了和基础矩阵类似的约束表达

$$p_r^T F p_l = 0 \quad (184)$$

只不过这时, 对应点已经变成像素坐标了。

由于极点对图像中的任意一个点都满足上述关系, 因此左右相机的极点满足下式

$$\begin{aligned} e_r^T F &= 0 \\ F e_l &= 0 \end{aligned} \quad (185)$$

1.5 单应矩阵和基础矩阵差异和联系

之前定义的单应矩阵也可以构建一个左右相机观测点的约束

$$p_r = H p_l \quad (186)$$

如果我们把上式稍微处理下, 两侧都**叉乘**一个 p_r , 有

$$[p_r_{\times}] H p_l = 0 \quad (187)$$

这个形式和上面的基础矩阵形式非常接近, 但是需要注意的是, 两者有很明显的区别:

1. 基础矩阵的形式是左边点**转置乘法**, 而单应矩阵的形式是**叉乘**
2. 几何上来说, 基础矩阵表达的是**点和线的约束关系**, 而单应矩阵是**点对点的约束**, 也就是我们说的单应性。
3. 基础矩阵本身并不依赖场景中的物体, 只和两帧图像的相对位姿和相机内参有关; 而单应矩阵不但和上述有关系, 还和特定的世界平面有关。
4. 单应矩阵的自由度为8; 基础矩阵的自由度只有7, 因为其本身是齐次矩阵, 且其需要满足 $\det(F) = 0$

5. F和H都可以通过某种方法来分解出对应的旋转和平移，但是当求解F的点都在一个平面上时，或者两帧图像发生了纯旋转时，E的自由度会退化，求解会不稳定，这个时候就更适合使用单应矩阵来求解

具体的分解过程我们不在这里细讲，对于标定来说，通常使用已知的标志物进行初始相对旋转和平移的计算。

实际上，我们知道，**两点的叉乘即是过两点的直线参数**，则有右极线方程

$$l_r = e_r \times x_r = e_r \times Hx_l = Fx_l \quad (188)$$

有

$$F = e_r \times H \quad (189)$$

1.6 双目/立体校正(Stereo Rectify)(Bouguet 校正)

我们知道，在双目视觉的流程中，需要找到左右相机的匹配点，从而使用三角法来恢复其深度。虽然在前面我们通过数学的方法，约束了匹配点可能存在的直线位置，但是在实际代码中如果沿着一个不规律的直线来搜索，匹配代价依然很高。



立体校正正式将实际成像变成理想共面的形式，在搜索时就可以只沿一个方向进行匹配。对于程序而言，内存是连续的，计算性能会更好。我们这里以主要的Bouguet方法来讲解。

1.6.1 再谈旋转矩阵

已知旋转矩阵 R 可以将坐标系 O_1 中的点坐标变换至坐标系 O_2 ，即满足如下形式

$$\begin{bmatrix} X_2 \\ Y_2 \\ Z_2 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix} \quad (190)$$

令

$$\begin{cases} r_1^T = (r_{11}, r_{21}, r_{31}) \\ r_2^T = (r_{12}, r_{22}, r_{32}) \\ r_3^T = (r_{13}, r_{23}, r_{33}) \end{cases} \quad (191)$$

则 $R = (r_1, r_2, r_3)$ ，我们取坐标系 O_1 中特殊的点坐标 $A(1, 0, 0)$ 、 $B(0, 1, 0)$ 、 $C(0, 0, 1)$ 代入上式，即旋转到坐标系 O_2 ，有

$$\begin{cases} P_{A2} = RP_{A1} = r_1 \\ P_{B2} = RP_{B1} = r_2 \\ P_{C2} = RP_{C1} = r_3 \end{cases} \quad (192)$$

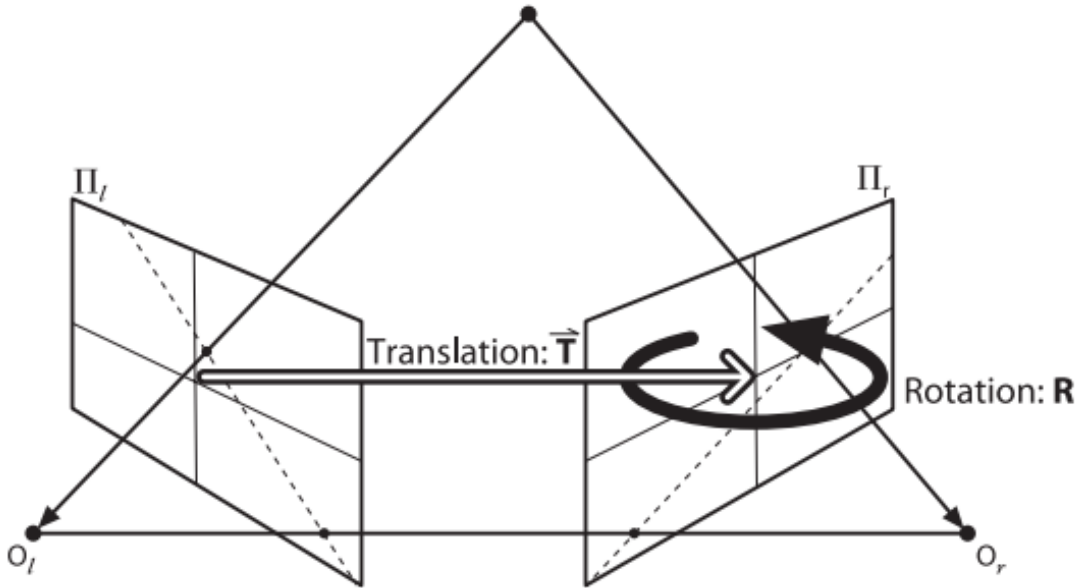
也就是说，坐标系 O_1 到坐标系 O_2 投影的旋转矩阵的列向量本质上为 O_1 的 XYZ 轴在 O_2 中的坐标。同样的，由于旋转矩阵的特性，其行向量本质上为 O_2 的 XYZ 轴在 O_1 中的坐标。

1.6.2 构建校正矩阵

假设我们通过标定知道了左右相机的相对旋转和平移 $R T$ ，即满足

$$P_r = RP_l + T \quad (193)$$

我们分几步把下图变成两侧成像平面都平行且行对准的状态，



1. 旋转成像平面和光轴平行

旋转矩阵本身就已经包含了两个成像平面的相对旋转关系，实际上，按照 R 直接来旋转左平面就可以达到效果。但是，如果两个相机的光轴夹角较大，那么就会导致左平面旋转过多，从而导致大量的画面损失，因此我们考虑使用折衷的方法进行处理，即两个相机各旋转一半。

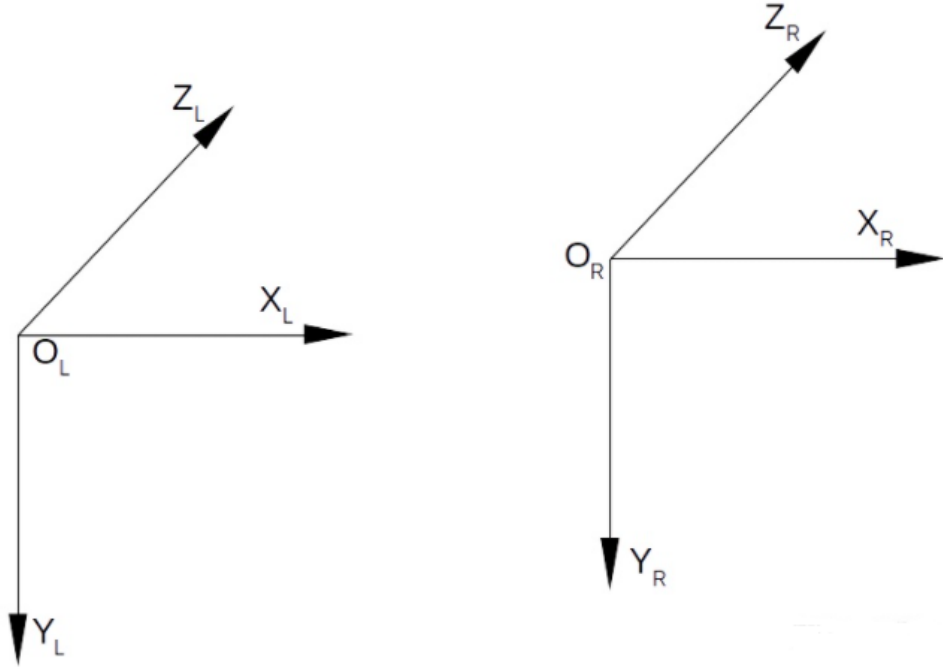
- 首先将旋转矩阵变换为罗德里格斯形式 r
- 将左相机按照正向旋转一半，其旋转为 $0.5r$ ，称为 r_l
- 将右相机逆向旋转一半，其旋转为 $-0.5r$ ，称为 r_r

需要注意的是，经过上述旋转，相机的平移向量 T 发生了变化，我们定义其为 t

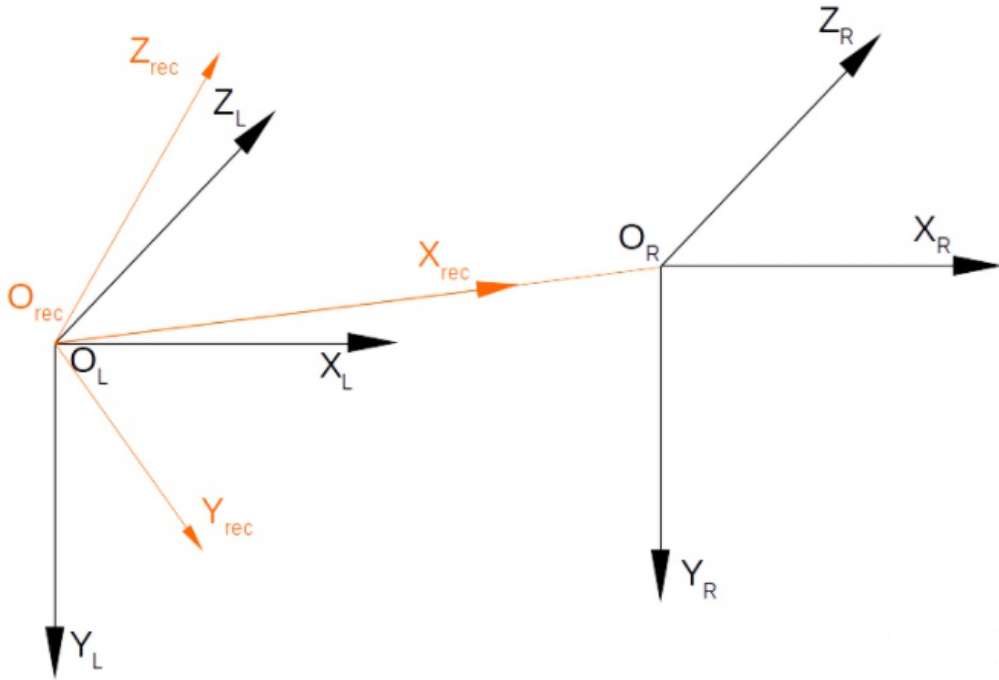
具体 t 的计算大家可以自行推导下

2. 计算行对准旋转矩阵 R_{rect}

经过上述的变化，我们实际上得到如下一种状态，



即处于一种面平行但是行不对准的情况，需要通过另外的旋转矩阵来处理，将其处理为



- 我们需要将极线对齐，因此选取极点方向作为第一个轴

$$\vec{e}_1 = \frac{\begin{matrix} \vec{} \\ -t \end{matrix}}{\| -t \|} \quad (194)$$

$$\vec{e}_2 = \frac{(0, 0, 1)^T \times \begin{matrix} \vec{} \\ -t \end{matrix}}{\|(0, 0, 1)^T \times \begin{matrix} \vec{} \\ -t \end{matrix}\|} = \frac{(y, -x, 0)}{\sqrt{t_y^2 + t_x^2}} \quad (195)$$

◦

这里因为我们的旋转和平移矩阵为从左到右的变换，因此 t 需要取反

- 第三个轴没什么要求，只要和第一第二个轴正交即可

$$\vec{e}_3 = \vec{e}_1 \times \vec{e}_2 \quad (196)$$

3. 由于左右相机的极线对准矩阵总是相同的，因此总的旋转矩阵就变为

$$\begin{aligned} R_l &= R_{rect} * r_l \\ R_r &= R_{rect} * r_r \end{aligned} \quad (197)$$

4. 计算新的投影矩阵

这部分比较简单，和去畸变部分比较类似，主要是新投影焦距和光心的一些处理（投影后相机焦距需要相等），我们具体结合代码来讲解。

2. 双目标定代码讲解

2.1 部分知识

2.1.1 相对旋转计算

如果我们知道两个相机相对世界坐标系的变化如

$$\begin{aligned} R_1 W + T_1 &= W_1 \\ R_2 W + T_2 &= W_2 \end{aligned} \quad (198)$$

则两个坐标系之间的变换关系为

$$R_2 R_1^T W_1 + (T_2 - R_2 R_1^T T_1) = W_2 \quad (199)$$